

Class Overview

In Class:

- Boundary Value Testing
- Break (10 minutes)?
- Equivalence Class Testing
- Break (10 minutes)?
- Path-Based Testing: Program Graph Coverage
- Break (10 minutes)
- Path-Based Testing: Code Coverage?

After Class: Office Hours

Homework:

- 2/4: Symbolic Execution
- 2/11: Code Coverage Inputs?

Readings: Chapter 8 (Code-Based Testing)

1

Extensions

- Extensions Rules
 - Not available after the due date
 - Not available after the last day of *this* class, not including the exam
 - Must be requested via e-mail
 - You may get early reviews during the extended period after the original due date
 - You may ask for an additional extension before an extended due date

2

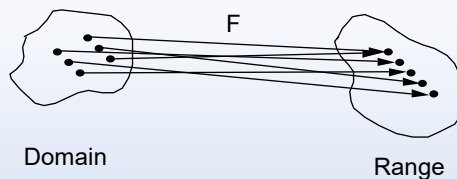
Boundary Value Testing

3

Functional Testing

The rationale for referring to specification-based testing as “functional testing” is likely due to the abstraction that any program can be viewed as a mapping from its Input Domain to its Output Range:

$$F(\text{Input}) = \text{Output}$$

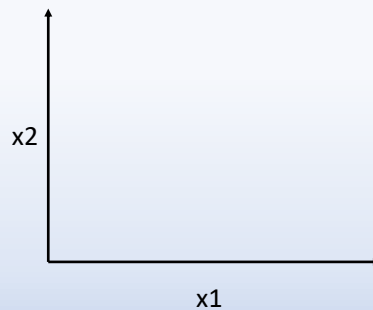


4

Functional Testing

Functional testing uses information about functional mappings to identify test cases.

- We illustrate much of this with a function F of two variables:
 - x_1 , and
 - x_2 .



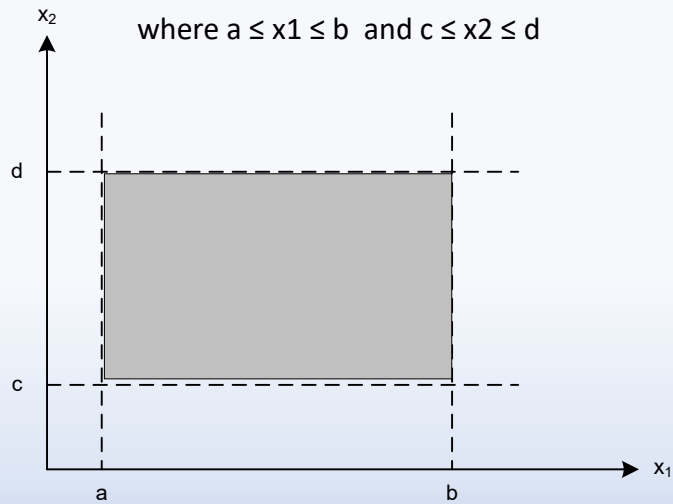
5

Rationale for Boundary Value Testing

- Industrial, commercial, and defense software all note that faults seem to be more prevalent when variables have values at or near their extreme boundaries.
- In the early 1990s, there was a commercial test tool, named simply "T", that implemented boundary value testing.
- T was originally requested by the U.S. Army
- The product has been merged into
 - Teamwork from Cadre Systems, and
 - Software through Pictures from Aonix

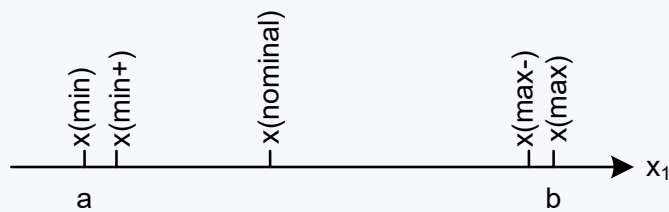
6

Input Domain of $F(x_1, x_2)$



7

Input Boundary Value Testing



- Test values for variable x , where
 - $a \leq x_1 \leq b$, and
 - $x(\min)$, $x(\min+)$, ... $x(\max)$ are the names from the T tool.

8

Boundary Value Testing

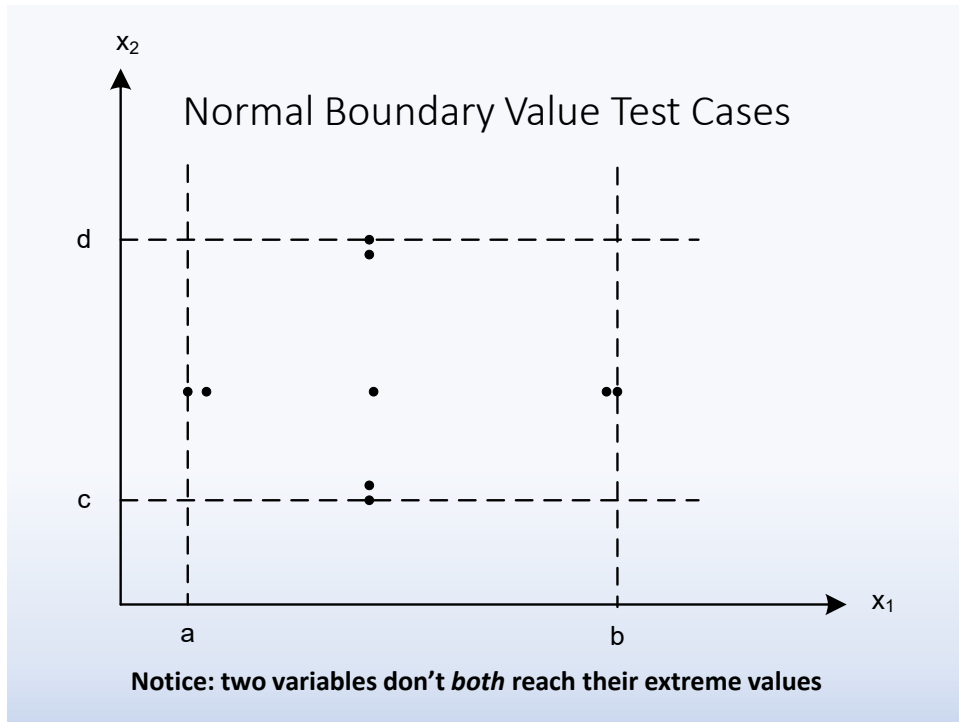
- Four kinds of boundary value testing:
 - **Normal** boundary value testing
 - **Robust** boundary value testing
 - **Worst case** boundary value testing
 - **Robust worst case** boundary value testing

9

Boundary Value Testing

- Four kinds of boundary value testing:
 - **Normal** boundary value testing
 - **Robust** boundary value testing
 - **Worst case** boundary value testing
 - **Robust worst case** boundary value testing

10



11

Normal Boundary Value Method

- Hold all variables at their nominal values.
- Let one variable assume its boundary values.
- Repeat this for each variable.
- This will (hopefully) reveal all faults that can be attributed to a single variable.

12

In Class Exercise!

- Given the following function specification, identify the **number of** normal boundary value test cases:

```
double weightedAverage(double weight1,
                       double average1,
                       double weight2,
                       double average2)
```

- The range of each variable is 0 to 500, inclusive.
- The function returns the average of two weighted values (e.g., sets of values that have already been averaged).
- Note:** There should be a pattern you can exploit!

13

In Class Exercise!

Test Case	Weight 1	Average 1	Weight 2	Average 2
1	250	250	250	250
2	500	250	250	250
3	499	250	250	250
4	0	250	250	250
5	1	250	250	250
6	250	500	250	250
7	250	499	250	250
8	250	0	250	250
9	250	1	250	250

14

In Class Exercise!

Test Case	Weight 1	Average 1	Weight 2	Average 2
10	250	250	500	250
11	250	250	499	250
12	250	250	0	250
13	250	250	1	250
14	250	250	250	500
15	250	250	250	499
16	250	250	250	0
17	250	250	250	1

15

In Class Exercise!

```

public static double weightedAverage(double weight1,
                                     double average1,
                                     double weight2,
                                     double average2) {

    if(weight1<0 || average1<0 ||
        weight2<0 || average2<0)
        throw new IllegalArgumentException();

    if(weight1>500 || average1>500 ||
        weight2>500 || average2>500)
        throw new IllegalArgumentException();

    return ((weight1 * average1) + (weight2 * average2)) / (weight1 + weight2);
}

```

16

Boundary Value Testing

- Four kinds of boundary value testing:
 - **Normal** boundary value testing
 - **Robust** boundary value testing
 - **Worst case** boundary value testing
 - **Robust worst case** boundary value testing

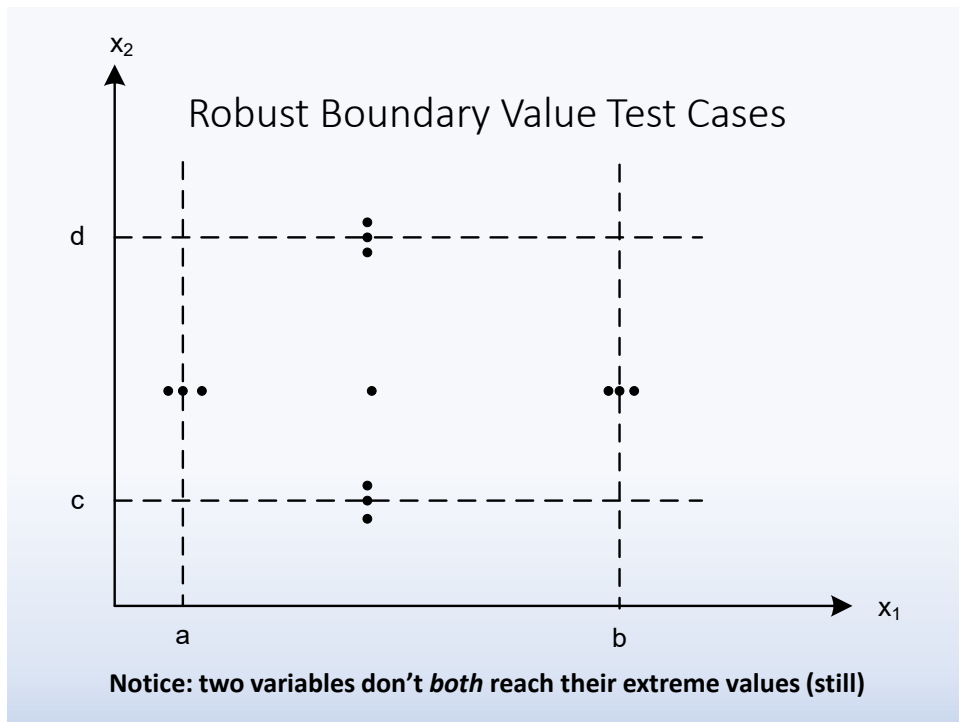
17

Robust Boundary Value Testing



- Stresses boundaries
- Advantages:
 - Find hidden functionality
 - Leads to exploratory testing
- Issues:
 - What are the expected outputs?
 - What if the programming language is strongly typed? (e.g., Ada)

18



19

In Class Exercise!

- Given the following function specification, identify the robust boundary value test cases that **cause an exception**:

```
double weightedAverage(double weight1,  
                        double average1,  
                        double weight2,  
                        double average2)
```

- The range of each variable is 0 to 500, inclusive.
- The function returns the average of two weighted values (e.g., sets of values that have already been averaged).

20

In Class Exercise!

```
public static double weightedAverage(double weight1,
                                     double average1,
                                     double weight2,
                                     double average2) {

    if(weight1<0 || average1<0 ||
        weight2<0 || average2<0)
        throw new IllegalArgumentException();

    if(weight1>500 || average1>500 ||
        weight2>500 || average2>500)
        throw new IllegalArgumentException();

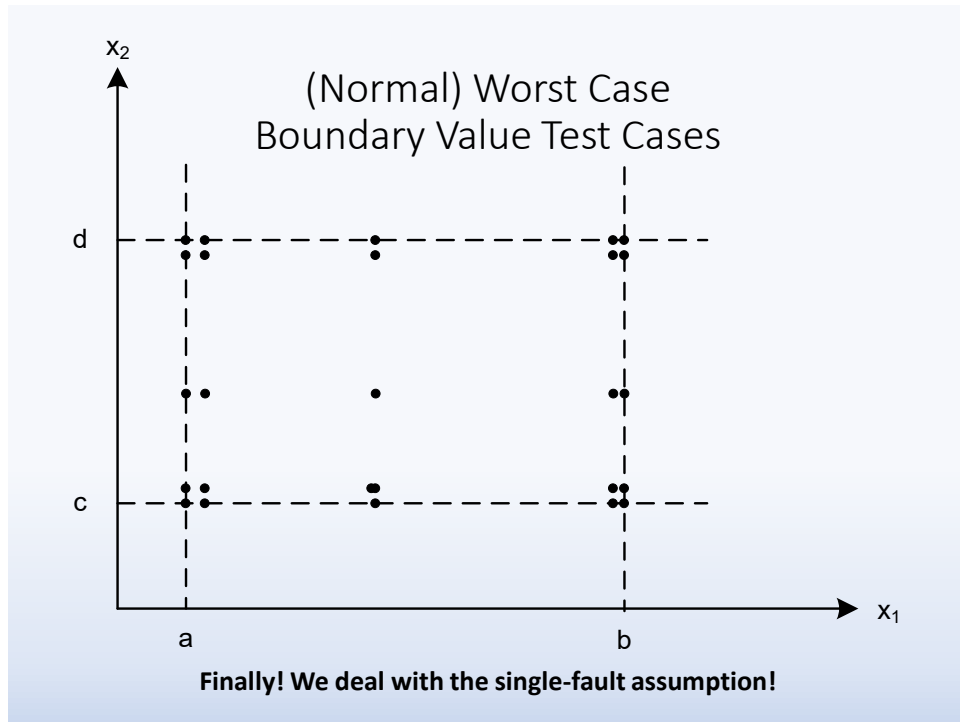
    return ((weight1 * average1) + (weight2 * average2)) / (weight1 + weight2);
}
```

21

Boundary Value Testing

- Four kinds of boundary value testing:
 - **Normal** boundary value testing
 - **Robust** boundary value testing
 - **Worst case** boundary value testing
 - **Robust worst case** boundary value testing

22



In Class Exercise!

- Given the following function specification, identify the worst case boundary value test cases that **discover a divide by zero**:

```
double weightedAverage(double weight1,  
                        double average1,  
                        double weight2,  
                        double average2)
```

- The range of each variable is 0 to 500, inclusive.
- The function returns the average of two weighted values (e.g., sets of values that have already been averaged).

In Class Exercise!

```
public static double weightedAverage(double weight1,
                                     double average1,
                                     double weight2,
                                     double average2) {

    if(weight1<0 || average1<0 ||
        weight2<0 || average2<0)
        throw new IllegalArgumentException();

    if(weight1>500 || average1>500 ||
        weight2>500 || average2>500)
        throw new IllegalArgumentException();

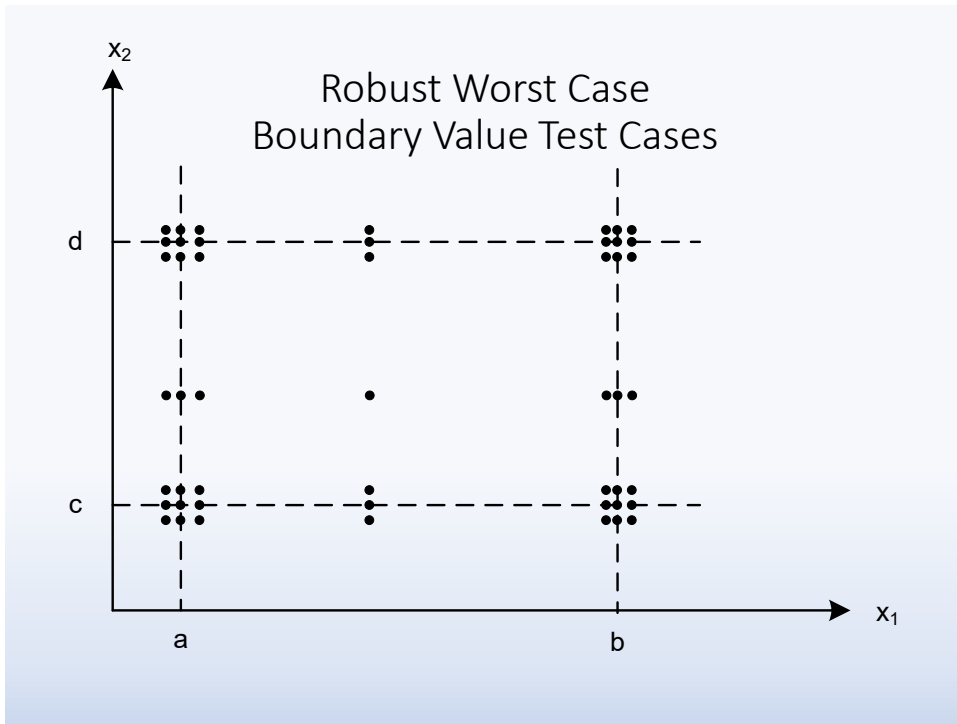
    return ((weight1 * average1) + (weight2 * average2)) / (weight1 + weight2);
}
```

25

Boundary Value Testing

- Four kinds of boundary value testing:
 - **Normal** boundary value testing
 - **Robust** boundary value testing
 - **Worst case** boundary value testing
 - **Robust worst case** boundary value testing

26



27

Boundary Value Testing

- So, if we have four kinds of boundary value testing:
 - **Normal** boundary value testing
 - **Robust** boundary value testing
 - **Worst case** boundary value testing
 - **Robust worst case** boundary value testing

How do we choose what to use?

28

Discussion

When would you use Boundary Value Testing?

29

How to choose?

- Two considerations apply to boundary value testing:
 - Are invalid values an issue?
 - Can we make the “single fault assumption” of reliability theory?
- Consequences of these considerations:
 - Invalid values require the robust choice
 - Multiplicity of faults requires worst case testing

	Valid Values	Valid and Invalid Values
Single Fault	Boundary Value	Robust Boundary Value
Multiple Fault	Worst Case Boundary Value	Robust Worst Case Boundary Value

30

Pros and Cons of Boundary Value Testing

- Advantages
 - Commercial tool support available
 - Easy to do/automate
 - Appropriate for calculation-intensive applications with variables that represent physical quantities (e.g., have units, such as meters, degrees, kilograms)
- Disadvantages
 - Inevitable potential for both gaps and redundancies
 - The gaps and redundancies can never be identified (specification-based)
 - Does not scale up well
 - Tools only generate inputs, user must generate expected outputs.

31



32

Is everyone's Clicker Working?

- **Yes / T** - Yes, my clicker is working!
- **No / F** – No, my clicker is **not** working!



33

Input vs Output?

- Could we apply boundary value testing to output values instead of just input values?
 - **Yes**, of course!
 - **No**, absolutely not!



34

Output Range Boundary Value Analysis

- Instead of covering the *input* or *domain*, we cover the *output* or *range*.
- What are some potential problems?
 - The whole range may not be achievable
 - We may not know what inputs provide the specific outputs
 - In many-to-one functions, we may not select the *right* input

35

Special Value Testing

- Appropriate for
 - complex mathematical calculations
 - worst case situations (similar to robustness)
 - problematic situations from past experience
- Characterized by...
 - "second guess" likely implementations
 - experience helps
 - frequently done by customer/user
 - defies measurement
 - highly intuitive
 - seldom repeatable
 - (and is often most effective)

Perhaps there is a better way to deal with these cases?

36